100
Pass

Akshat Agarwal

PDF generated at: 18 Feb 2026 12:17:14 UTC

View this report on HackerRank

Score

100% • 175.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 63 min 47 sec on 18 Feb 2026 16:40:38 IST

Candidate Information

Email	mr.akshatagarwal26@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	18 Feb 2026 16:40:38 IST
Time taken	63 min 47 sec/ 90 min
Work Experience	5 years
Invited by	Vipul
Invited on	18 Feb 2026 08:59:15 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	100%

2	Spring Boot Intermediate	100%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	100%	Hard	100%
TDD	100%	Spring	100%
JPA	100%	Spring Data	100%
Transactional	100%	Medium	100%
Graphs	100%	Java	100%
Exceptions	100%	OOP	100%

Questions

Spring Boot (Advanced) • 25 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	15 sec	✓

2	Response Multiple Choice	Spring Boot (Advanced)	5/5	-	27 sec	✓
3	Spring Data Transactional Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 23 sec	✓
4	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	45 sec	✓
5	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	5/5	-	55 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Minimize Path Value Coding	Problem Solving (Intermediate)	75/75	-	17 min 7 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: Vending Machine Coding	Java (Intermediate)	75/75	-	41 min 58 sec	✓

1. Specification

✓ Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● Specification ✓
- ○ Criteria
- ○ Query

No comments.

---

2. Response

Correct

Multiple Choice | Hard | Spring Boot | TDD

Question description

Which response will cause the following test case to fail?

```
@Test
   @DisplayName("Filter Price Check 3")
   public void FilterPriceCheck3() throws Exception {
      mockMvc.perform(get("/filter/price/150/500"))
            .andDo(print())
            .andExpect(jsonPath("$[*].product", containsInAnyOrder("Product 1", "Product 2" , "Produc
t 5")))
   }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] [{"product" : "Product 5"} , {"product" : "Product 1"}, {"product" : "Product 2"}]

- [ ] [{"product" : "Product 5"} , {"product" : "Product 2"}, {"product" : "Product 1"}]

- [ ] [{"product" : "Product 1"} , {"product" : "Product 2"}, {"product" : "Product 5"}]

- [x] **[{"product" : "product 5"} , {"product" : "product 2"}, {"product" : "product 1"}]**

No comments.

3. Spring Data Transactional

Correct

Multiple Choice | Spring | Spring Boot | JPA | Spring Data | Transactional | Medium

Question description

Given the following code, what happens when *StudentRepsitoryTest* runs?

Lesson.java

```
@Entity
public class Lesson {
   @Id
   private Long id;
   private String name;
   private Integer courseLength;
   private Integer courseLevel;
   @JsonBackReference
   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "student_id")
   private Student student;
   private boolean inProgress;
   //getter & setters
}
```

Student.java

```
@Entity
public class Student {
   @Id
   private Long id;
   private String name;
   private String surname;
   @JsonManagedReference
   @OneToMany(mappedBy = "student", fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST,
CascadeType.REFRESH})
   private List<Lesson> lessons;
   //getter & setters
}
```

StudentRepository.java

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long>{
    //Y
    Optional<Student> findById(Long id);
}
```

LessonRepository.java

```
@Repository
public interface LessonRepository extends JpaRepository<Lesson, Long> {
}
```

StudentService.java

```
@Service
public class StudentService{
    private StudentRepository studentRepository;
    public StudentService(StudentRepository studentRepository, LessonRepository lessonRepository)
{
        this.studentRepository = studentRepository;
        this.lessonRepository = lessonRepository;
    }

    public void save(Student student) {
        studentRepository.save(student);
    }

    //other crud metods

    //X
    public void deleteStudentWithLessons(Long id) throws SQLException {
        Student student =
                studentRepository.findById(id).orElseThrow(() -> new RuntimeException("Student not foun
d"));
        List<Lesson> lessons = student.getLessons();
        lessons.stream().forEach(lesson -> lesson.setStudent(null));
        studentRepository.delete(student);
```

```
        boolean inProgress =lessons.stream()
                .map(Lesson::isInProgress)
                .filter(progress -> progress == true)
                .findFirst()
                .orElseGet(() -> false);
        if (inProgress) {
            throw new SQLException("Cannot delete all Lessons. There are lessons in progress state");
        }
        lessonRepository.deleteAll(lessons);
    }
}
```

StudentRepositoryTest.java

```
@SpringBootTest
class StudentRepositoryTest {
    @Autowired
    StudentRepository studentRepository;
    @Autowired
    StudentService studentService;
    @Autowired
    LessonRepository lessonRepository;
    Student student;
    @BeforeEach
    public void init() {
        student = Student.builder().name("Jorn").surname("Deynhoven").build();
        studentRepository.save(student);
        lessonRepository.saveAll(List.of(
                Lesson.builder().name("Music").courseLength(10).student(student).courseLevel(123).build
(),
                Lesson.builder().name("Art").courseLength(20).courseLevel(123).student(student).build(),
                Lesson.builder().name("Trance").courseLength(30).courseLevel(123).student(student).inPro
gress(true).build()
        ));
    }
    @Test
    void deleteStudentWithLessonsTest() throws SQLException {
        studentService.deleteStudentWithLessons(student.getId());
    }
}
```

Interviewer guidelines

Candidate has to know how to use spring data in any spring application which uses spring data. If an entity wanted to remove from database without removing the child attributes, then first relation between them should be removed. Without doing this, spring will throw DataIntegrityViolationException to keep database consistent. To achive that without changing existing code, we need to annotate deleteStudentWithLessons(Long id) method with @Transactional annotation. With this spring will update the database states of child entities. In this case it will update Lesson attributes to have student as null. Then student will be removed in the next line.

After this there is a inProgress check for existing lessons. it will throw SQLException which is a checked exception. @Transactional annotation in default only rolls back for unchecked exception such as RuntimeException. So all changes will be reflected to database.

Option A and E is right. More info can be found in https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] ** It will try to delete the student then the DataIntegrityViolationException will be thrown. **

- [ ] It will delete the student successfully then an SQLException will be thrown with the message "Cannot delete all Lessons. There are lessons in progress state".

- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException and all database changes will be rolled back.

- [ ] It will throw a RuntimeException with the message "Student not found".

- [x] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException.

No comments.

4. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ( ) protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }

- ( ) protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }

- [x] protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }

- [ ] None of the above

No comments.

5. Spring Scopes

Correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Both are used to define the scope of spring beans.
- [x] @Scope(“request”) creates a bean instance for a single HTTP request.
- [x] @RequestScope can be used instead of @Scope(“request”).

@Scope("prototype") will return a different instance every time it is requested from the container.

No comments.

6. Minimize Path Value

Correct

Coding Graphs Medium

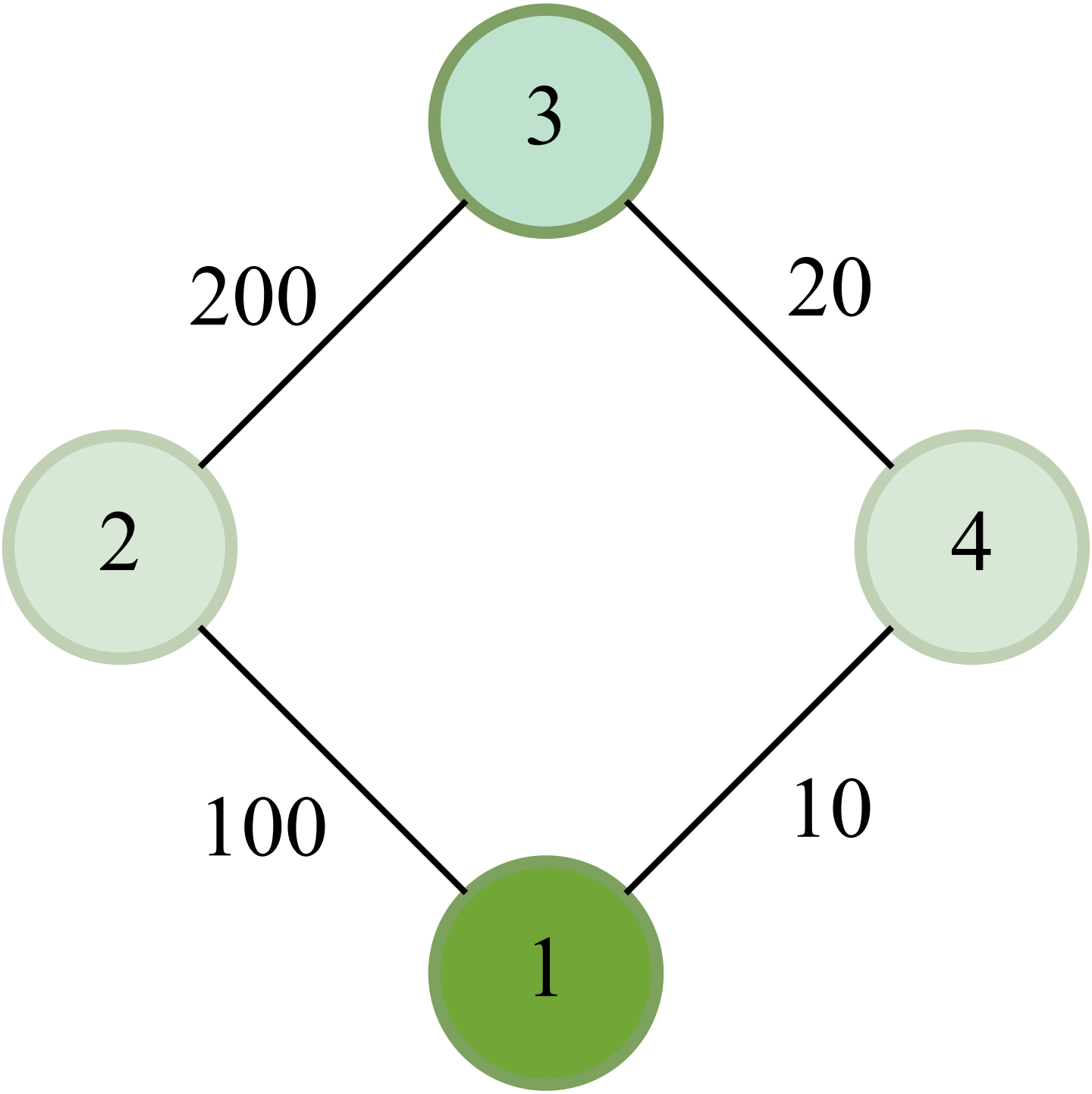
Question description

You have a weighted undirected graph with *graph_nodes* nodes and *m* edges. The stress level of a path between two nodes is defined as the weight of the heaviest edge in that path.

Given a source node *source* and a destination node *destination*, find the minimum possible stress level of a path. If no such path exists, return -1.

Example

Consider the following example below, where *source=1* and *destination=3.*



There are two paths, from node 1 to node 3.

1. The max weighted edge/stress level in path 1 -> 2 -> 3 is 200.
2. The max weighted edge/stress level in the path 1 -> 4 -> 3 is 20.

Return 20, the lower stress level from the second path.

Function Description

Complete the function *getMinimumStress* in the editor with the following parameter(s):

int graph_nodes: number of nodes
int graph_from[m]: one end node for an edge
int graph_to[m]: one end node for an edge
int graph_weight[m]: the weights of the edges
int source: the source node
int destination: the destination node

Returns

int: the value of the least stressful path, or -1 if no path exists

Note: Even though edge endpoints are called *graph_from[i]* and *graph_to[i]*, this is an undirected graph.

Constraints

- $1 \le graph_nodes \le 10^5$
- $1 \le |graph_from| = |graph_to| = |graph_weight| \le 10^5$ *(Where |A| denotes the size of array A)*
- $1 \le graph_from[i], graph_to[i], source, destination \le graph_nodes$
- $0 \le graph_weight[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains two space-separated integers *graph_nodes* and *m,* the number of nodes, and the number of edges of the graph.
Each of the following *m* lines contains three space-separated integers *u, v,* and *w,* meaning there is an edge connecting the nodes *u* and *v* of weight *w.*
The next line contains a single integer, denoting the index of the "*source*" node.
The next line contains a single integer, denoting the index of the "*destination*" node.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

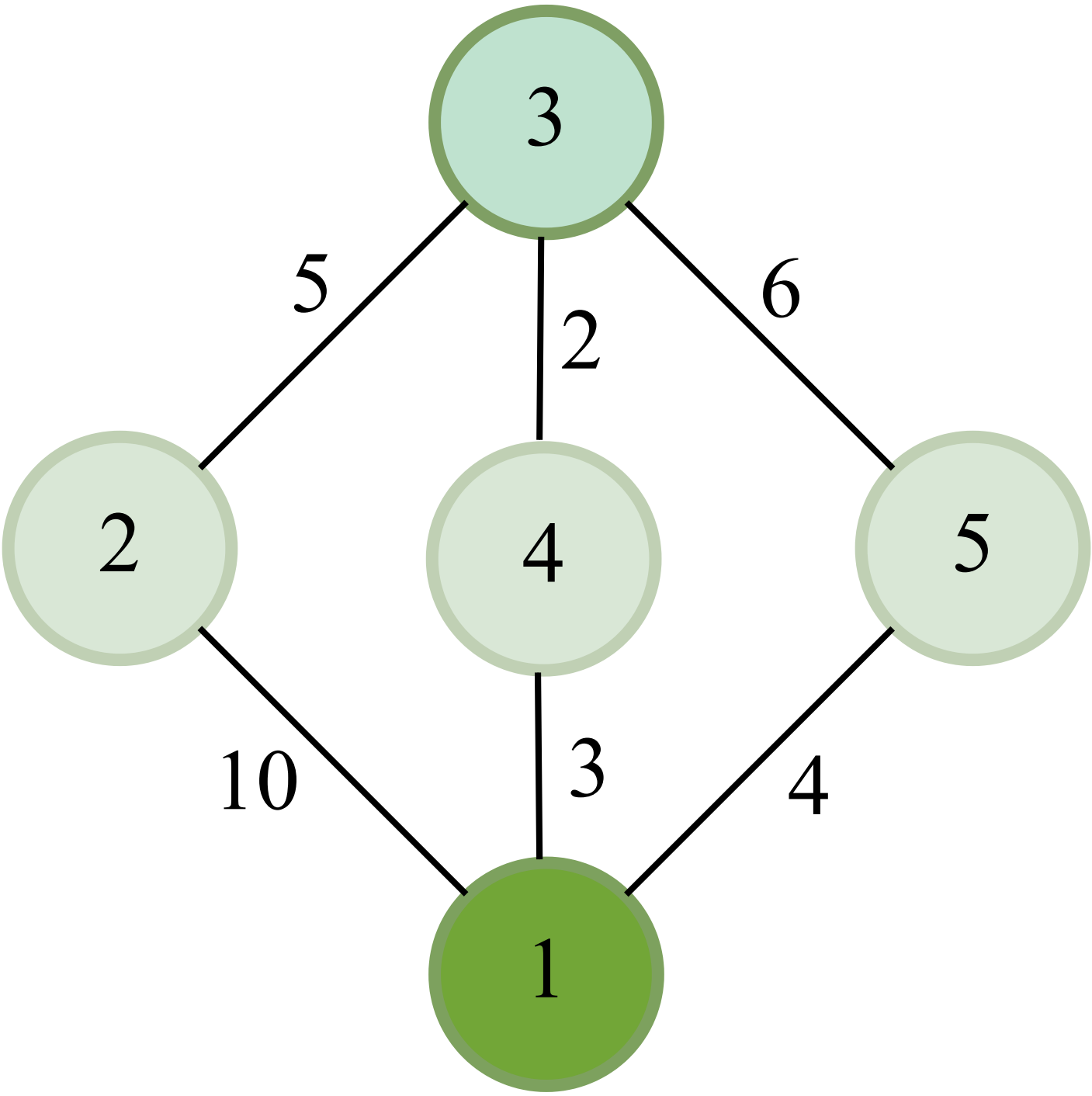
```
STDIN       FUNCTION
-----       ---------
5 6         graph_nodes = 5, m = 6
1 2 10      u = 1, v = 2, w = 10
```

```
2 3 5
1 4 3
4 3 2
1 5 4
5 3 6
1          source = 1
3          destination = 3
```

Sample Output

```
3
```

Explanation



There are three paths, from node 1 to node 3 :

1. 1 -> 2 -> 3. Edges have weights 10 and 5, the maximum weighted edge, and the stress level is 10.
2. 1 -> 4 -> 3. Edges have weights 3 and 2, and the stress level = 3.
3. 1 -> 5 -> 3. Edges have weights 4 and 6, and the stress level = 6.

▼ SAMPLE CASE 1

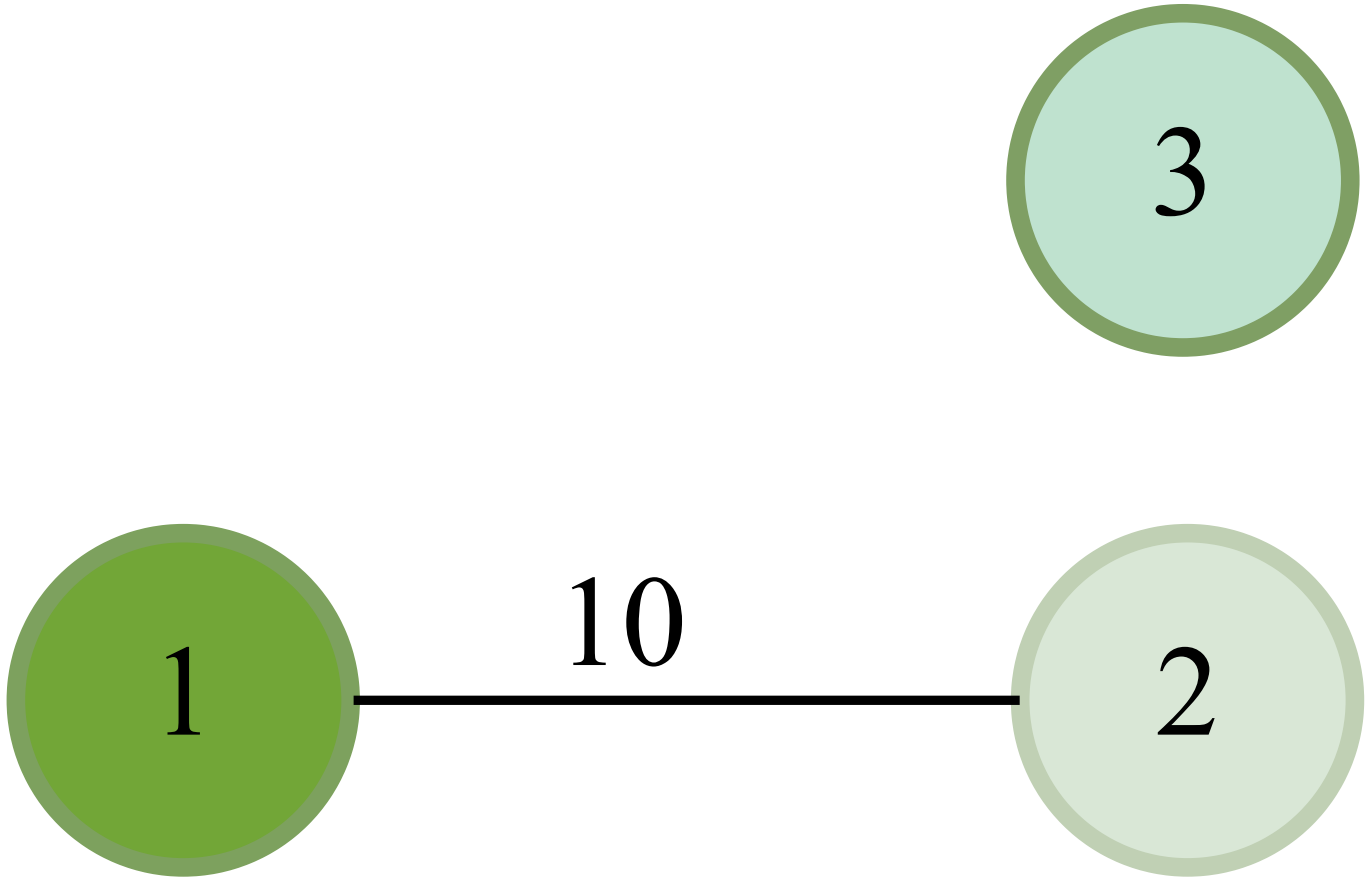
Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
3 1          graph_nodes = 3, m = 1
1 2 10       a = 1, b = 2, w = 10
1           source = 1
3           destination = 3
```

Sample Output

```
-1
```

Explanation



There is no path, from node 1 to node 3.

Interviewer guidelines

▼ SOLUTION

Python Solution

```
sys.setrecursionlimit(10**5)


def getMinimumStress(graph_nodes, graph_from, graph_to, graph_weight, source, destination):

    adj = [[] for i in range(graph_nodes + 1)]
    vis = [False] * (graph_nodes + 1)

    for x, y, w in zip(graph_from, graph_to, graph_weight):
        adj[x].append((y, w))
        adj[y].append((x, w))

    def dfs(node, mx):
        vis[node] = True
        if node == destination:
            return True
        ans = False
        for i, w in adj[node]:
            if w <= mx and not vis[i]:
                ans |= dfs(i, mx)
        return ans

    if not dfs(source, max(graph_weight)):
        return -1

    lo, hi = 0, max(graph_weight)
    while hi - lo > 1:
        vis = [False] * (graph_nodes + 1)
        mid = (hi + lo) // 2
        if dfs(source, mid):
            hi = mid
        else:
            lo = mid
    return hi
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
```

```java
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;



class Result {

    /*
     * Complete the 'getMinimumStress' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     *  1. WEIGHTED_INTEGER_GRAPH graph
     *  2. INTEGER source
     *  3. INTEGER destination
     */

    /*
     * For the weighted graph, <name>:
     *
     * 1. The number of nodes is <name>Nodes.
     * 2. The number of edges is <name>Edges.
     * 3. An edge exists between <name>From[i] and <name>To[i]. The weight of the edge is <name>Weight[i].
     *
     */

    public static int getMinimumStress(int graphNodes,
    List<Integer> graphFrom,
    List<Integer> graphTo,
    List<Integer> graphWeight,
    int source,
    int destination)
    {
        List<int[]>[] graph = new ArrayList[graphNodes+1];
        for(int i=1; i<=graphNodes; i++) {
            graph[i] = new ArrayList<>();
        }

        for(int i=0; i<graphFrom.size(); i++) {
            int u = graphFrom.get(i);
            int v = graphTo.get(i);
            int w = graphWeight.get(i);

```

```
            graph[u].add(new int[]{v, w});
            graph[v].add(new int[]{u, w});
        }

        int[] dist = new int[graphNodes + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[source] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));

        pq.add(new int[]{source, 0});

        while(!pq.isEmpty()) {
            int[] cur = pq.poll();
            int node = cur[0];
            int stress = cur[1];

            if(node == destination) {
                return stress;
            }

            for(int[] ni : graph[node]) {
                int next = ni[0];
                int weight = ni[1];

                int newStress = Math.max(stress, weight);

                if(newStress < dist[next]) {
                    dist[next] = newStress;
                    pq.add(new int[]{next, newStress});
                }
            }
        }
        return -1;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

```

```
        String[] graphNodesEdges = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int graphNodes = Integer.parseInt(graphNodesEdges[0]);
        int graphEdges = Integer.parseInt(graphNodesEdges[1]);

        List<Integer> graphFrom = new ArrayList<>();
        List<Integer> graphTo = new ArrayList<>();
        List<Integer> graphWeight = new ArrayList<>();

        IntStream.range(0, graphEdges).forEach(i -> {
            try {
                String[] graphFromToWeight = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

                graphFrom.add(Integer.parseInt(graphFromToWeight[0]));
                graphTo.add(Integer.parseInt(graphFromToWeight[1]));
                graphWeight.add(Integer.parseInt(graphFromToWeight[2]));
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });

        int source = Integer.parseInt(bufferedReader.readLine().trim());

        int destination = Integer.parseInt(bufferedReader.readLine().trim());

        int result = Result.getMinimumStress(graphNodes, graphFrom, graphTo, graphWeight, source, destination);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED

TestCase 0	Easy	Sample	Success	1	0.0543 sec	37.6 KB
TestCase 1	Easy	Sample	Success	1	0.0501 sec	37.7 KB
TestCase 2	Easy	Sample	Success	1	0.0539 sec	37.7 KB
TestCase 3	Easy	Hidden	Success	6	0.6374 sec	113 KB
TestCase 4	Easy	Hidden	Success	6	0.4193 sec	103 KB
TestCase 5	Easy	Hidden	Success	6	0.6102 sec	113 KB
TestCase 6	Easy	Hidden	Success	6	0.6066 sec	124 KB
TestCase 7	Easy	Hidden	Success	6	0.6144 sec	132 KB
TestCase 8	Easy	Hidden	Success	6	0.5934 sec	113 KB
TestCase 9	Easy	Hidden	Success	6	0.477 sec	119 KB
TestCase 10	Easy	Hidden	Success	6	0.6727 sec	120 KB
TestCase 11	Easy	Hidden	Success	6	0.6901 sec	116 KB
TestCase 12	Easy	Hidden	Success	6	0.2771 sec	75.2 KB

TestCase 13	Easy	Hidden	Success	6	0.2985 sec	77.8 KB
TestCase 14	Easy	Hidden	Success	6	0.4564 sec	126 KB

No comments.

7. Java: Vending Machine

Correct

Coding Medium Java Exceptions OOP

Question description

Design a vending machine that serves 3 drinks: "Coke", "Fanta", and "Sprite".

Create a *VendingMachine* class that implements the following methods:

- *void registerDrink(int buttonNumber, Drink drink)*: Registers a type of drink with a particular *buttonNumber* of the vending machine. Here, *Drink* is an interface provided in the locked stub.
- *ServedDrinkSummary(int buttonNumber, int money)*: Returns the Drink that is served when *buttonNumber* is pressed, and an amount of money is entered by a customer.

The class contains two attributes:

- *change*: the units of currency to return to the consumer
- *drink*: the drink requested

There are three potential error conditions:

- If the machine has run out of the drink, an *OutOfStockException* should be thrown with the message "{drinkName} is out of stock".
- If the money the user enters is insufficient to buy the drink, an *InsufficientMoneyException* should be thrown.
- If the drink is out of stock and the user has not entered enough money, throw an *OutOfStockException* with the previously mentioned message.

The *OutOfStockException, InsufficientMoneyException*, and *ServedDrinkSummary* classes are provided in the locked stub.

The locked stub validates the *VendingMachine* class by processing some requests.

Constraints

- $1 \le totalNumberOfRequests \le 10^4$
- $0 \le buttonNumber \le 2$
- $0 \le money \le 100$
- $0 \le quantity$ available of each $drink \le 10^4$
- $0 \le price$ per unit of each $drink \le 10^4$
- No two drinks are assigned the same *buttonNumber.*

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains three integers that represent the *quantity* of *Coke*, *Fanta,* and *Sprite* in the vending machine respectively.
The second line contains three integers that represent the *price* per unit of *Coke*, *Fanta,* and *Sprite* respectively.
The third line contains three integers that represent the *buttonNumbers* assigned to *Coke*, *Fanta,* and *Sprite* respectively.
The fourth line contains an integer, *totalNumberOfRequests .*
The next *totalNumberOfRequests* lines contain the *buttonNumber* pressed and *money* inserted separated by a space.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
2 1 0    quantity Coke = 2, Fanta = 1, Sprite = 0
5 10 15  price Coke = 5, Fanta = 10, Sprite = 15
0 1 2    buttonNumbers Coke = 0, Fanta = 1, Sprite = 2
4        totalNumberOfRequests = 4
0 20     buttonNumber = 0, money = 20 for the first request
2 20     buttonNumber = 2, money = 20 for the second request
2 5      ...
1 5
```

Sample Output

```
Vending machine set up
Coke 15
Sprite is out of stock
Sprite is out of stock
Insufficient money
```

Explanation

- *0 20:* A request to process a can of *Coke* and *20* units of money is provided to the vending machine. *Coke* is in stock and costs *5* units of money so *15* units in change is returned to the consumer. The locked stub prints "*Coke 15".*
- *2 20:* A request to process a can of *Sprite* and *20* units of money is provided to the vending machine. *Sprite* is out of stock so an *OutOfStockException* is thrown, and the stub prints "*Sprite is out of stock".*
- *2 5:* A request to process a can of *Sprite* and *5* units of money is provided to the vending machine. *Sprite* is out of stock and the amount of *money* is insufficient so an *OutOfStockException* is thrown and the stub prints, "*Sprite is out of stock".*
- *1 5:* A request to process a can of *Fanta* and *5* units of money is provided. Since the price of a unit of *Fanta is 10* and it is in stock, an *InsufficientMoneyException* is thrown, and the stub prints "*Insufficient money".*

Interviewer guidelines

▼ SOLUTION

Solution for Java 7, 8 and 15

```
class VendingMachine{
    HashMap<Integer, Drink> drinkRelatedToButton = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink){
        drinkRelatedToButton.put(buttonIdx, drink);
    }

    ServeDrinkSummary dispatch(int buttonPressed, int money)throws OutOfStockException, Insuffi
cientMoneyException{
        ServeDrinkSummary serveDrinkSummary = new ServeDrinkSummary();
        serveDrinkSummary.servedDrink = drinkRelatedToButton.get(buttonPressed);
        serveDrinkSummary.change = money - serveDrinkSummary.servedDrink.getPrice();
```

```
        if(serveDrinkSummary.servedDrink.getQuantityLeft() == 0){
            throw new OutOfStockException(serveDrinkSummary.servedDrink.getName() + " is out of st
ock");
        }

        if(serveDrinkSummary.change < 0){
            throw new InsufficientMoneyException();
        }
        return serveDrinkSummary;
    }
}
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

interface Drink{
    int getPrice();
    String getName();
    int getQuantityLeft();
    void serveDrink();
}

class ServeDrinkSummary{
    Drink servedDrink;
    int change;
}

class OutOfStockException extends Exception{
    public OutOfStockException(String message){
        super(message);
    }
}

class InsufficientMoneyException extends Exception{
    public InsufficientMoneyException(){
        super();
    }
}
import java.io.*;
import java.util.*;
```

```
import java.text.*;
import java.math.*;
import java.util.regex.*;

interface Drink{
    int getPrice();
    String getName();
    int getQuantityLeft();
    void serveDrink();
}

class ServeDrinkSummary{
    Drink servedDrink;
    int change;
}

class OutOfStockException extends Exception{
    public OutOfStockException(String message){
        super(message);
    }
}

class InsufficientMoneyException extends Exception{
    public InsufficientMoneyException(){
        super();
    }
}
class VendingMachine{

    private Map<Integer, Drink> drinkMap = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink) {
        drinkMap.put(buttonIdx, drink);
    }

    ServeDrinkSummary dispatch(int buttonPressed, int money)
        throws OutOfStockException, InsufficientMoneyException{

            Drink drink = drinkMap.get(buttonPressed);

            if(money < drink.getPrice() && drink.getQuantityLeft() == 0) {
                throw new OutOfStockException(drink.getName() + " is out of stock");
            }

            if(money < drink.getPrice()) {
```

```
                throw new InsufficientMoneyException();
            }

            if(drink.getQuantityLeft() == 0) {
                throw new OutOfStockException(drink.getName() + " is out of stock");
            }


            int change = money - drink.getPrice();

            ServeDrinkSummary serveDrinkSummary = new ServeDrinkSummary();
            serveDrinkSummary.servedDrink = drink;
            serveDrinkSummary.change = change;
            return serveDrinkSummary
            ;
    }
}


class Solution{

    public static void main(String[] args) throws IOException{
        String arr[];
        int quantity[] = new int[3];
        int pricePerUnit[] = new int[3];
        int buttonAssigned[] = new int[3];

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            quantity[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            pricePerUnit[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            buttonAssigned[i] = Integer.parseInt(arr[i]);
        }
```

```java

        VendingMachine vendingMachine = new VendingMachine();
        Drink coke = new Coke(pricePerUnit[0], "Coke", quantity[0]);
        Drink fanta = new Fanta(pricePerUnit[1], "Fanta", quantity[1]);
        Drink sprite = new Sprite(pricePerUnit[2], "Sprite", quantity[2]);
        vendingMachine.registerDrink(buttonAssigned[0], coke);
        vendingMachine.registerDrink(buttonAssigned[1], fanta);
        vendingMachine.registerDrink(buttonAssigned[2], sprite);
        out.println("Vending machine set up");

        int totalNumberOfRequests = Integer.parseInt(br.readLine().trim());
        while(totalNumberOfRequests-- > 0){
            arr = br.readLine().split(" ");
            int buttonPressed = Integer.parseInt(arr[0]),
                money = Integer.parseInt(arr[1]);
            try{
                ServeDrinkSummary serveDrinkSummary = vendingMachine.dispatch(buttonPressed, money);
                serveDrinkSummary.servedDrink.serveDrink();
                out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
            }catch(OutOfStockException e){
                out.println(e.getMessage());
            }catch(InsufficientMoneyException e){
                out.println("Insufficient money");
            }
        }

        out.flush();
        out.close();
    }
}

class Coke implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Coke(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
```

```
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Fanta implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Fanta(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Sprite implements Drink{
```

```
    private int price;
    private String name;
    private int quantityLeft;

    public Sprite(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Solution{

    public static void main(String[] args) throws IOException{
        String arr[];
        int quantity[] = new int[3], pricePerUnit[] = new int[3], buttonAssigned[] = new int[3];

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            quantity[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
```

```
            pricePerUnit[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            buttonAssigned[i] = Integer.parseInt(arr[i]);
        }

        VendingMachine vendingMachine = new VendingMachine();
        Drink coke = new Coke(pricePerUnit[0], "Coke", quantity[0]);
        Drink fanta = new Fanta(pricePerUnit[1], "Fanta", quantity[1]);
        Drink sprite = new Sprite(pricePerUnit[2], "Sprite", quantity[2]);
        vendingMachine.registerDrink(buttonAssigned[0], coke);
        vendingMachine.registerDrink(buttonAssigned[1], fanta);
        vendingMachine.registerDrink(buttonAssigned[2], sprite);
        out.println("Vending machine set up");

        int totalNumberOfRequests = Integer.parseInt(br.readLine().trim());
        while(totalNumberOfRequests-- > 0){
            arr = br.readLine().split(" ");
            int buttonPressed = Integer.parseInt(arr[0]),
                money = Integer.parseInt(arr[1]);
            try{
                ServeDrinkSummary serveDrinkSummary = vendingMachine.dispatch(buttonPressed, money);
                serveDrinkSummary.servedDrink.serveDrink();
                out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
            }catch(OutOfStockException e){
                out.println(e.getMessage());
            }catch(InsufficientMoneyException e){
                out.println("Insufficient money");
            }
        }

        out.flush();
        out.close();
    }
}

class Coke implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Coke(int price, String name, int quantityLeft){
```

```
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Fanta implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Fanta(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

```

```
    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Sprite implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Sprite(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0524 sec	37.6 KB

TestCase 1	Easy	Hidden	Success	6	0.0467 sec	37 KB
TestCase 2	Easy	Hidden	Success	6	0.0708 sec	39.3 KB
TestCase 3	Easy	Hidden	Success	6	0.0688 sec	38.9 KB
TestCase 4	Medium	Hidden	Success	10	0.0786 sec	39 KB
TestCase 5	Medium	Hidden	Success	10	0.1112 sec	50.5 KB
TestCase 6	Medium	Hidden	Success	10	0.1704 sec	50.8 KB
TestCase 7	Hard	Hidden	Success	13	0.1402 sec	49.6 KB
TestCase 8	Hard	Hidden	Success	13	0.1637 sec	49.7 KB

No comments.